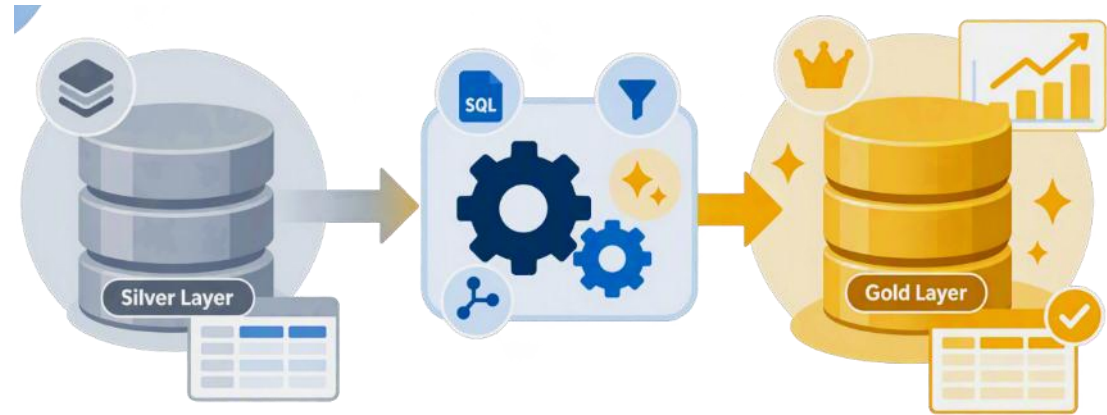


Gold Layer Transformations



Gold Layer: Analytics-Ready with Star Schema

Business-ready data: Silver tables reshaped into a **star schema** for dashboards, reports and analytics

Fact Tables : records what happened

It contains quantitative, **numeric measures**. e.g., sales, revenue, quantity sold, cost

Dimension Tables : provide **context** for analyzing the facts by answering:

Who? Customer, Employee

What? Product, Category

When? Date, Month, Year

Where? Store, City, Country

How? Sales Channel, Payment Method

Why it is useful

Business-friendly : analyze data from different perspectives (customer, product, time, and location)

Simple : facts and dimensions are easy to understand and query

Fast : minimizes expensive join operations and facilitates faster aggregations

Consistent : business metrics (e.g., revenue and profit) are defined once and used everywhere

Easy to explore : answer any metric by asking Who? What? When? Where? How?

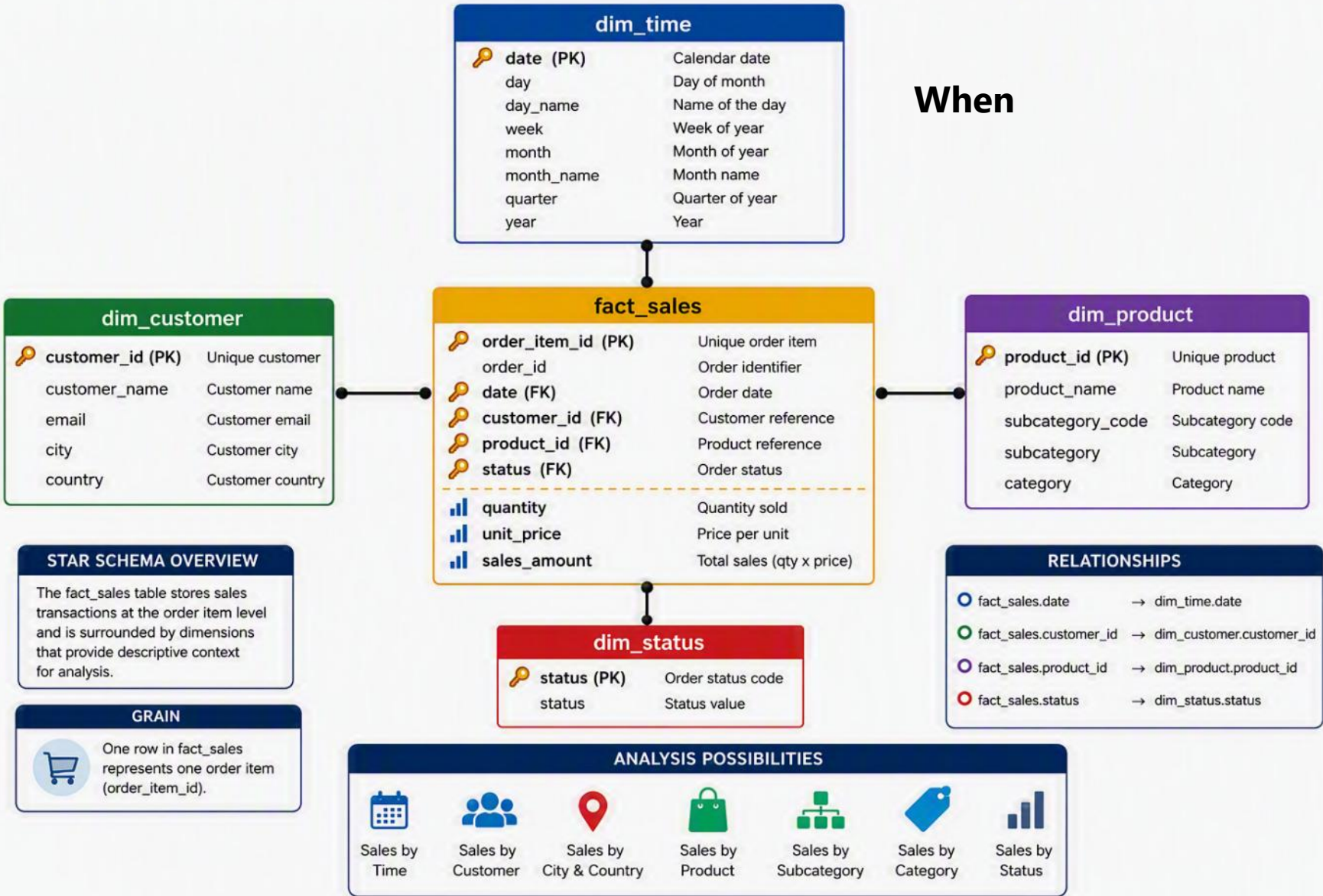
Facts tell us WHAT happened. Dimensions explain WHO, WHAT, WHEN, WHERE and HOW it happened.

Star Schema Example

Who
&
Where

When

What



Build a star schema in three steps



1 Define the fact grain What does one row represent?

- e.g., One fact row = one order line.
- Keep every measure at that grain. (e.g., do not mix order-line records with daily totals in one fact table)



2 Create dimensions and their hierarchies What can we group it by?

- Give each dimension a **unique key** and include the attributes users will *filter* or *group by*.
- Define its **hierarchy** (roll-up path)

E.g., Product dimension:

Product → Subcategory → Category

Running Shoe → Athletic Footwear → Footwear



3 Create the fact table What can we measure?

numeric measures (e.g., quantity and sales_amount)
+ **foreign keys** that link to the dimension tables

One fact_sales row: dimension keys provide context, and measures provide the numbers



Key principle: One fact table, one grain. Use a separate fact table for data at a different grain

Dimensions and their hierarchies



The `dim_product` hierarchy lets analysts sum `sales_amount` by **product**, then roll it up to **subcategory** or **category**, while `fact_sales` remains at the order-line grain.



The `dim_location` hierarchy lets analysts sum `sales_amount` by **city**, then roll it up to **country** to the **gcc**, while `fact_sales` remains at the order-line grain.

Star schema enables analytics operations: rollup, drilldown, slice, and dice



Key takeaway

The level of aggregation changes, but the fact table remains at the same grain (e.g., one order line).



Supported by hierarchies Works for product and location hierarchies (and others).

Step 2: Build two dimensions in SQL

Customer gives the “who”; Date supplies calendar context for every sale.

CUSTOMER dimension:

```
CREATE TABLE IF NOT EXISTS gold.dim_customer (  
    customer_id INTEGER,  
    customer_name VARCHAR,  
    city VARCHAR  
);  
MERGE INTO gold.dim_customer AS t  
USING silver.customers AS s  
    ON t.customer_id = s.customer_id  
WHEN MATCHED THEN UPDATE SET  
    customer_name = s.customer_name,  
    city = s.city  
WHEN NOT MATCHED THEN INSERT  
    (customer_id, customer_name, city)  
VALUES  
    (s.customer_id, s.customer_name, s.city);
```

DATE dimension

```
CREATE TABLE IF NOT EXISTS gold.dim_time (  
    date DATE, year INTEGER, month INTEGER  
);  
MERGE INTO gold.dim_time AS t  
USING (  
    SELECT DISTINCT order_date AS date,  
        year(order_date) AS year,  
        month(order_date) AS month  
    FROM silver.orders  
    WHERE order_date IS NOT NULL  
        AND order_date <= DATE '2026-09-18'  
) AS s ON t.date = s.date  
WHEN NOT MATCHED THEN INSERT  
    (date, year, month)  
VALUES (s.date, s.year, s.month);
```

The example calls the Date dimension dim_time. Its unique date is the join key.

Step 2: Build two dimensions in Pandas

Load the current Gold tables first, then apply a customer upsert and new dates.

STEP 2 / PANDAS

CUSTOMER: latest values win

```
customer_delta = silver_customers[
    ["customer_id", "customer_name", "city"]
]

# Existing Gold rows come first.
dim_customer = pd.concat(
    [dim_customer, customer_delta],
    ignore_index=True
).drop_duplicates(
    "customer_id", keep="last"
)
```

DATE: derive calendar fields

```
dates = pd.to_datetime(
    silver_orders["order_date"]
).dropna()
dates = dates[
    dates.le(pd.Timestamp(run_date))
].drop_duplicates()

date_delta = pd.DataFrame({"date": dates})
date_delta["year"] = date_delta["date"].dt.year
date_delta["month"] = date_delta["date"].dt.month
dim_time = pd.concat(
    [dim_time, date_delta], ignore_index=True
).drop_duplicates("date")
```

Step 3: Load the sales fact in SQL

Join Silver order items to orders, calculate the measure, then upsert by item ID.

STEP 3 / SQL

```
MERGE INTO gold.fact_sales AS t
USING (
  SELECT i.order_item_id,
         o.order_date AS date, o.customer_id,
         i.quantity, i.unit_price,
         i.quantity * i.unit_price AS sales_amount
  FROM silver.order_items AS i
  JOIN silver.orders AS o
    ON i.order_id = o.order_id
  WHERE o.order_date <= DATE '2026-09-18'
) AS s ON t.order_item_id = s.order_item_id
WHEN MATCHED THEN UPDATE SET
  date = s.date,
  customer_id = s.customer_id,
  quantity = s.quantity,
  unit_price = s.unit_price,
  sales_amount = s.sales_amount
WHEN NOT MATCHED THEN INSERT
  (order_item_id, date, customer_id,
   quantity, unit_price, sales_amount)
VALUES (s.order_item_id, s.date, s.customer_id,
        s.quantity, s.unit_price, s.sales_amount);
```

Table setup and keys

Create gold.fact_sales once. Use order_item_id for the grain; date and customer_id link the two dimensions.

Incremental behavior

Existing lines update. New lines insert. Re-running the same load does not append duplicates.

Measure: sales_amount = quantity × unit_price, at one row per order item.

Step 3: Load the sales fact in Pandas

Use the same line-item grain, date and customer keys, and calculated sales amount.

STEP 3 / PANDAS

```
as_of = silver_orders.loc[
    silver_orders["order_date"] <= pd.Timestamp(run_date),
    ["order_id", "order_date", "customer_id"]
]
lines = silver_order_items.merge(
    as_of, on="order_id", how="inner",
    validate="many_to_one"
)
assert lines["order_item_id"].is_unique
lines["sales_amount"] = (
    lines["quantity"] * lines["unit_price"]
)
fact_delta = lines.rename(
    columns={"order_date": "date"}
)[["order_item_id", "date", "customer_id",
    "quantity", "unit_price", "sales_amount"]]

fact_sales = pd.concat(
    [fact_sales, fact_delta], ignore_index=True
).drop_duplicates("order_item_id", keep="last")
```

Before loading

Check that customer_id and date exist in their dimensions, with matching types.

After loading

Persist the result as an atomic Gold update.
The DataFrame alone is not a stored table.

Keep one fact row per order_item_id, even when an existing line changes.